

# OSSP Skill Week 9 - Job Control and Signal Delivery

## Task 1: List Active Jobs

### Aim

To create a simple job table, display the active jobs with their process IDs and states, and update the state after a job completes.

### Program

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <sys/wait.h>

#define MAX_JOBS 5

struct Job {
    int id;
    pid_t pid;
    char state[20];
};

static void list_jobs(const struct Job jobs[], int count)
{
    for (int i = 0; i < count; i++)
        printf("[%d] PID: %d State: %s\n", jobs[i].id, jobs[i].pid, jobs[i].state);
}

int main(void)
{
    struct Job jobs[MAX_JOBS];
    int job_count = 0;

    pid_t pid = fork();
    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        sleep(5);
        exit(0);
    }

    jobs[job_count].id = job_count + 1;
    jobs[job_count].pid = pid;
    snprintf(jobs[job_count].state, sizeof(jobs[job_count].state), "Running");
    job_count++;

    printf("myShell> jobs\n");
    printf("\n--- Active Jobs ---\n");
    list_jobs(jobs, job_count);

    if (waitpid(pid, NULL, 0) < 0) {
        perror("waitpid");
        return 1;
    }

    snprintf(jobs[0].state, sizeof(jobs[0].state), "Completed");

    printf("\n--- Updated Job Status ---\n");
    list_jobs(jobs, job_count);

    return 0;
}
```

## Output

```
hrushika@Hrushikas-MacBook-Air pjsdj % ./jobs
myShell> jobs

--- Active Jobs ---
[1] PID: 71337 State: Running

--- Updated Job Status ---
[1] PID: 71337 State: Completed
hrushika@Hrushikas-MacBook-Air pjsdj %
```

## Observation

The active job was listed with its job ID, process ID and the state Running. After the child process completed, the listing showed the same job with the state Completed.

## Result

Active job listing and job state tracking were implemented successfully.

## Task 2: Foreground Job Control

### Aim

To create a child process, run it in the foreground, wait for it to finish and update its job state.

### Program

```
foreground.c > main(void)
1  #include <stdio.h>
2  #include <unistd.h>
3  #include <stdlib.h>
4  #include <sys/wait.h>
5
6  int main(void)
7  {
8      printf("Starting foreground job...\n");
9      fflush(stdout);
10
11     pid_t pid = fork();
12     if (pid < 0) {
13         perror("fork");
14         return 1;
15     }
16
17     if (pid == 0) {
18         printf("Child process running in foreground. PID: %d\n", getpid());
19         sleep(3);
20         printf("Foreground job completed.\n");
21         exit(0);
22     }
23
24     printf("Job started with PID: %d\n", pid);
25     printf("Transferring execution to foreground...\n");
26
27     /* The parent blocks until the foreground job ends */
28     if (waitpid(pid, NULL, 0) < 0) {
29         perror("waitpid");
30         return 1;
31     }
32
33     printf("Foreground job finished.\n");
34     printf("Job state updated to Completed.\n");
35     return 0;
36 }
```

## Output

```
hrushika@Hrushikas-MacBook-Air pjsdj % gcc foreground.c -o
hrushika@Hrushikas-MacBook-Air pjsdj % ./foreground
Starting foreground job...
Job started with PID: 71510
Transferring execution to foreground...
Child process running in foreground. PID: 71510
Foreground job completed.
Foreground job finished.
Job state updated to Completed.
hrushika@Hrushikas-MacBook-Air pjsdj %
```

## Observation

The child process was created with `fork()`, and the parent waited for it with `waitpid()`. The shell could not continue until the target process finished, which is how a foreground job behaves.

## Result

Foreground job execution and completion tracking were implemented successfully.

## Task 3: Signal Delivery

### Aim

To demonstrate signal delivery by handling the `SIGINT` and `SIGTERM` signals.

### Program

```
#include <stdio.h>
#include <signal.h>
#include <string.h>
#include <unistd.h>

/* Only async-signal-safe calls (write, _exit) are used in the handler */
static void handle_signal(int sig)
{
    if (sig == SIGINT) {
        const char msg[] = "\nSIGINT received. Process is still running.\n";
        write(STDOUT_FILENO, msg, sizeof(msg) - 1);
    } else if (sig == SIGTERM) {
        const char msg[] = "\nSIGTERM received. Terminating process...\n";
        write(STDOUT_FILENO, msg, sizeof(msg) - 1);
        _exit(0);
    }
}

int main(void)
{
    struct sigaction sa;
    memset(&sa, 0, sizeof(sa));
    sa.sa_handler = handle_signal;
    sigemptyset(&sa.sa_mask);

    if (sigaction(SIGINT, &sa, NULL) < 0 || sigaction(SIGTERM, &sa, NULL) < 0) {
        perror("sigaction");
        return 1;
    }

    printf("Signal handling program started.\n");
    printf("PID: %d\n", getpid());
    printf("Press Ctrl+C to send SIGINT.\n");

    while (1) {
        printf("Process is running...\n");
        sleep(2);
    }

    return 0;
}
```

## Output

```
Job state updated to Completed.
hrushika@Hrushikas-MacBook-Air pjsdj % gcc signals.c -o signals
hrushika@Hrushikas-MacBook-Air pjsdj % ./signals
Signal handling program started.
PID: 71689
Press Ctrl+C to send SIGINT.
Process is running...
Process is running...
Process is running...
Process is running...
Process is running...
Process is running...
Process is running...
Process is running...
^C
SIGINT received. Process is still running.
Process is running...
Process is running...
Process is running...
^C
SIGINT received. Process is still running.
Process is running...
^C
SIGINT received. Process is still running.
Process is running...
^C
SIGINT received. Process is still running.
Process is running...
Process is running...
Process is running...
Process is running...
Process is running...
Process is running...
Process is running...
Process is running...
```

## Observation

When Ctrl+C was pressed, the terminal delivered SIGINT and the registered handler ran instead of the default action, so the process kept running. SIGTERM ran the handler, which printed its message and ended the process.

## Result

Signal delivery and signal handling were demonstrated successfully.

## Task 4: Resume Stopped Jobs

### Aim

To stop a child process with SIGSTOP, resume it with SIGCONT and update its job state.

### Program

```
resume.c: 7 int main(void)
1  #include <stdio.h>
2  #include <unistd.h>
3  #include <signal.h>
4  #include <stdlib.h>
5  #include <sys/wait.h>
6
7  int main(void)
8  {
9      int status;
10
11      pid_t pid = fork();
12      if (pid < 0) {
13          perror("fork");
14          return 1;
15      }
16
17      if (pid == 0) {
18          printf("Child process started. PID: %d\n", getpid());
19          for (int i = 1; i <= 5; i++) {
20              printf("Child running: step %d\n", i);
21              fflush(stdout);
22              sleep(2);
23          }
24          printf("Child process completed.\n");
25          exit(0);
26      }
27
28      printf("Job started with PID: %d\n", pid);
29      fflush(stdout);
30      sleep(2);
31
32      printf("\nSending SIGSTOP...\n");
33      if (kill(pid, SIGSTOP) < 0) {
34          perror("kill SIGSTOP");
35          return 1;
36      }
37
38      /* Confirm that the child has actually stopped */
39      if (waitpid(pid, &status, WUNTRACED) > 0 && WIFSTOPPED(status))
40          printf("Job is now stopped.\n");
41
42      sleep(2);
43
44      printf("\nSending SIGCONT...\n");
45      if (kill(pid, SIGCONT) < 0) {
46          perror("kill SIGCONT");
47          return 1;
48      }
49
50      printf("Job resumed successfully.\n");
51      fflush(stdout);
52
53      waitpid(pid, &status, 0);
54      printf("Job completed.\n");
55
56      return 0;
57  }
```

## Output

```
hcrushika@Hrushikas-MacBook-Air pjsdj % ./resume
Job started with PID: 72417
Child process started. PID: 72417
Child running: step 1
Child running: step 2

Sending SIGSTOP...
Job is now stopped.

Sending SIGCONT...
Job resumed successfully.
Child running: step 3
Child running: step 4
Child running: step 5
Child process completed.
Job completed.
hcrushika@Hrushikas-MacBook-Air pjsdj %
```

## Observation

The child printed step 1, was stopped by SIGSTOP and printed nothing while stopped. After SIGCONT it continued from step 2 to step 5, and the parent reported completion once it exited.

## Result

Recovery of a stopped job using SIGCONT and process state management were implemented successfully.